ACCESSMAP+ DATABASE PROJECT

Final Report

Muskan & Harshitha Seetharaman
CS5200: Database Management Systems Project
Fall 2025

==========

TABLE OF CONTENTS

1. README: Creating and Running the Project
2. Technical Specifications
3. Conceptual Design (UML Diagram)
4. Logical Design (Reverse Engineering Diagram)
5. Final User Flow
6. Lessons Learned
7. Future Work

==========

1. README: CREATING AND RUNNING THE PROJECT

1.1 Prerequisites

Before you begin, ensure you have the following software installed on your machine:

1. MySQL Server 8.0 or higher
   • Download from: https://dev.mysql.com/downloads/mysql/
   • Installation: Follow the installation wizard and set a root password

2. MySQL Workbench (Optional but recommended)
   • Download from: https://dev.mysql.com/downloads/workbench/
   • Used for database management and visualization

3. Command Line Interface (CLI)
   • Windows: Command Prompt or PowerShell
   • macOS/Linux: Terminal

1.2 Database Setup Instructions

Step 1: Download the Database Dump File

- Locate the file: AccessMapPlus_FINAL_DUMP.sql
- Save it to a known directory on your computer

Step 2: Import the Database

Method A: Using MySQL Command Line

1. Open your terminal or command prompt
2. Navigate to the directory containing the dump file:
   cd /path/to/dump/file

3. Import the database:
   mysql -u root -p < AccessMapPlus_FINAL_DUMP.sql

4. Enter your MySQL root password when prompted

Method B: Using MySQL Workbench

1. Open MySQL Workbench
2. Connect to your MySQL server instance
3. Go to: Server > Data Import
4. Select "Import from Self-Contained File"
5. Browse and select: AccessMapPlus_FINAL_DUMP.sql
6. Click "Start Import"
7. Wait for the import to complete

Step 3: Verify Installation

1. Connect to MySQL:
   mysql -u root -p

2. Switch to the database:
   USE AccessMapPlus;

3. Verify tables are created:
   SHOW TABLES;

   Expected output: 11 tables
   - User
   - Authority
   - Place

- Accessibility_Feature
- Feature_Instance
- Review
- Review_Media
- Issue_Report
- Issue_Media
- Maintenance_Responsibility

4. Verify stored procedures:
   SHOW PROCEDURE STATUS WHERE Db = 'AccessMapPlus';

   Expected output: 5 procedures

5. Verify functions:
   SHOW FUNCTION STATUS WHERE Db = 'AccessMapPlus';

   Expected output: 3 functions

6. Verify triggers:
   SHOW TRIGGERS;

   Expected output: 3 triggers

7. Verify events:
   SHOW EVENTS;

   Expected output: 2 events

1.3 Sample Data Verification

The database comes pre-populated with sample data. To verify:

```
SELECT COUNT(*) FROM User;          -- Should return 5
SELECT COUNT(*) FROM Authority;     -- Should return 3
SELECT COUNT(*) FROM Place;         -- Should return 6
SELECT COUNT(*) FROM Review;        -- Should return 5
SELECT COUNT(*) FROM Issue_Report;  -- Should return 4
```

1.4 Testing Database Functionality

Test 1: Call a Stored Procedure

```
CALL proc_add_review(1, 1, 5, 'Test review', @review_id, @status);
SELECT @review_id, @status;
```

Test 2: Use a Function

```
SELECT get_average_rating(1) AS AverageRating;
```

Test 3: View Sample Data

```
SELECT * FROM Place WHERE verified = TRUE;
```

1.5 Installation Directory Structure

Expected directory structure:

```
project_root/
├── AccessMapPlussql    (Database dump file)
├── H_M_final_report.md                    (This documentation)
```

1.6 System Requirements

- Operating System: Windows 10/11, macOS 10.14+, or Linux (Ubuntu 18.04+)
- RAM: Minimum 4GB (8GB recommended)
- Disk Space: Minimum 500MB free space
- MySQL Version: 8.0 or higher
- Network: Not required (local database)

1.7 Troubleshooting

Common Issues and Solutions:

Issue 1: "Access denied for user 'root'"
Solution: Verify your MySQL password is correct

Issue 2: "Database already exists"
Solution: The dump file drops and recreates the database automatically

Issue 3: "Events not running"
Solution: Enable event scheduler:

```
SET GLOBAL event_scheduler = ON;
```

Issue 4: "Unknown database 'AccessMapPlus'"
Solution: Ensure the dump file executed completely without errors

1.8 Additional Resources

- MySQL Documentation: https://dev.mysql.com/doc/
- MySQL Tutorial: https://www.mysqltutorial.org/
- Project Documentation: See DATABASE_DOCUMENTATION.md

==========

2. TECHNICAL SPECIFICATIONS

2.1 Database Management System (DBMS)

MySQL 8.0
- Primary database system used for the AccessMap+ project
- Chosen for its reliability, performance, and widespread industry adoption
- Supports ACID properties ensuring data integrity
- Provides robust transaction management and concurrency control

2.2 Database Design Tools

MySQL Workbench 8.0
- Used for database schema design and visualization
- Utilized for reverse engineering the logical design
- Employed for creating and testing SQL queries
- Facilitated the export of database dumps

2.3 SQL Language Features Utilized

Data Definition Language (DDL)
- CREATE DATABASE: Database creation
- CREATE TABLE: Table definitions with constraints
- CREATE INDEX: Performance optimization indexes
- ALTER TABLE: Schema modifications (if needed)
- DROP DATABASE: Clean database reset

Data Manipulation Language (DML)

- INSERT: Sample data population
- UPDATE: Data modification operations
- DELETE: Data removal operations
- SELECT: Data retrieval queries

Data Control Language (DCL)

- GRANT: User permission management
- REVOKE: Permission removal

Transaction Control Language (TCL)

- START TRANSACTION: Begin atomic operations
- COMMIT: Confirm changes
- ROLLBACK: Revert changes on error

2.4 Advanced Database Features

Stored Procedures (5 total)

1. proc_create_issue
   - Purpose: Create accessibility issue reports with validation
   - Parameters: user_id, place_id, issue_type, severity, description
   - Returns: report_id, status message
   - Features: Transaction management, error handling

2. proc_assign_responsibility
   - Purpose: Assign maintenance responsibilities to authorities
   - Parameters: authority_id, instance_id, role, since_date
   - Returns: status message
   - Features: Automatic place_id lookup, duplicate handling

3. proc_add_review
   - Purpose: Add or update user reviews for places
   - Parameters: user_id, place_id, rating, comment
   - Returns: review_id, status message
   - Features: Rating validation, duplicate key handling

4. proc_resolve_issue
   - Purpose: Mark issue reports as resolved
   - Parameters: report_id, authority_id
   - Returns: status message

- Features: Automatic timestamp setting

5. proc_update_feature_status
   - Purpose: Update accessibility feature status
   - Parameters: instance_id, status
   - Returns: status message
   - Features: Last checked timestamp update

User-Defined Functions (3 total)

1. get_average_rating(place_id)
   - Returns: DECIMAL(3,2) - Average rating for a place
   - Handles NULL values with COALESCE

2. get_open_issue_count(place_id)
   - Returns: INT - Count of open/in-progress issues
   - Filters by status: 'Open', 'In Progress'

3. count_working_features(place_id)
   - Returns: INT - Count of working features
   - Filters by status: 'Working'

Triggers (3 total)

1. trg_before_insert_review
   - Event: BEFORE INSERT on Review
   - Purpose: Validate rating is between 1-5
   - Action: SIGNAL error if invalid

2. trg_before_update_issue_report
   - Event: BEFORE UPDATE on Issue_Report
   - Purpose: Auto-set resolved_at timestamp
   - Action: Set timestamp when status changes to 'Resolved'

3. trg_after_update_feature_instance
   - Event: AFTER UPDATE on Feature_Instance
   - Purpose: Update place's last modified timestamp
   - Action: Update Place.updated_at when status changes

Events (2 total)

1. ev_auto_close_old_resolved_issues
   - Schedule: Every 1 day

- Purpose: Auto-close resolved issues after 30 days
- Action: Update status from 'Resolved' to 'Closed'

2. ev_maintenance_reminder
   - Schedule: Every 1 week
   - Purpose: Flag features needing inspection
   - Action: Update last_checked for features not checked in 90 days

2.5 Constraints and Data Integrity

Primary Keys
- All 11 tables have auto-incrementing primary keys
- Ensures unique identification of records
- Provides indexing for performance

Foreign Keys with Referential Actions
- ON DELETE CASCADE: Child records deleted when parent deleted
  Example: Deleting a User deletes their Reviews

- ON DELETE RESTRICT: Prevents deletion if children exist
  Example: Cannot delete Accessibility_Feature if in use

- ON UPDATE CASCADE: Updates propagate to child records
  Applied to all foreign key relationships

Field Constraints
- NOT NULL: Required fields (first_name, email, password_hash, etc.)
- UNIQUE: Prevents duplicates (email in User and Authority)
- DEFAULT: Auto-values (created_at, status fields)
- CHECK: Data validation (rating 1-5, latitude/longitude ranges)
- ENUM: Limited value sets (severity, status, role)

Composite Constraints
- UNIQUE(user_id, place_id, created_at): Prevent duplicate reviews
- UNIQUE(authority_id, place_id, instance_id): Prevent duplicate responsibilities
- CHECK(resolved_at >= created_at): Logical date validation

2.6 Performance Optimization

Indexing Strategy (20+ indexes)

- Primary key indexes (automatic)
- Foreign key indexes for JOIN performance
- Composite indexes for common query patterns
  Example: INDEX(city, category) for location-based searches
- Status field indexes for filtering
- Timestamp indexes for date range queries

Query Optimization Techniques
- Use of stored procedures to reduce network overhead
- Efficient JOIN operations via indexed foreign keys
- COALESCE for NULL handling in aggregate functions
- Strategic use of ENUM for limited value sets

2.7 Error Handling Implementation

Transaction Management
- START TRANSACTION: Begin atomic operations
- COMMIT: Confirm successful operations
- ROLLBACK: Revert on error or validation failure

Exception Handlers
- EXIT HANDLER FOR SQLEXCEPTION: Catch all SQL errors
- Custom error messages returned via OUT parameters
- Validation checks before database modifications

Error Response Pattern
All procedures return status messages:
- SUCCESS: Operation completed successfully
- ERROR: Specific error description (e.g., "User does not exist")

2.8 Normalization

Third Normal Form (3NF) Compliance
- First Normal Form (1NF): All attributes are atomic
- Second Normal Form (2NF): No partial dependencies
- Third Normal Form (3NF): No transitive dependencies

Example of Normalization:
Instead of storing feature details repeatedly in Feature_Instance:
- Accessibility_Feature: Stores feature types and descriptions

- Feature_Instance: References feature_id, stores only status

This eliminates redundancy and update anomalies.

2.9 Security Features

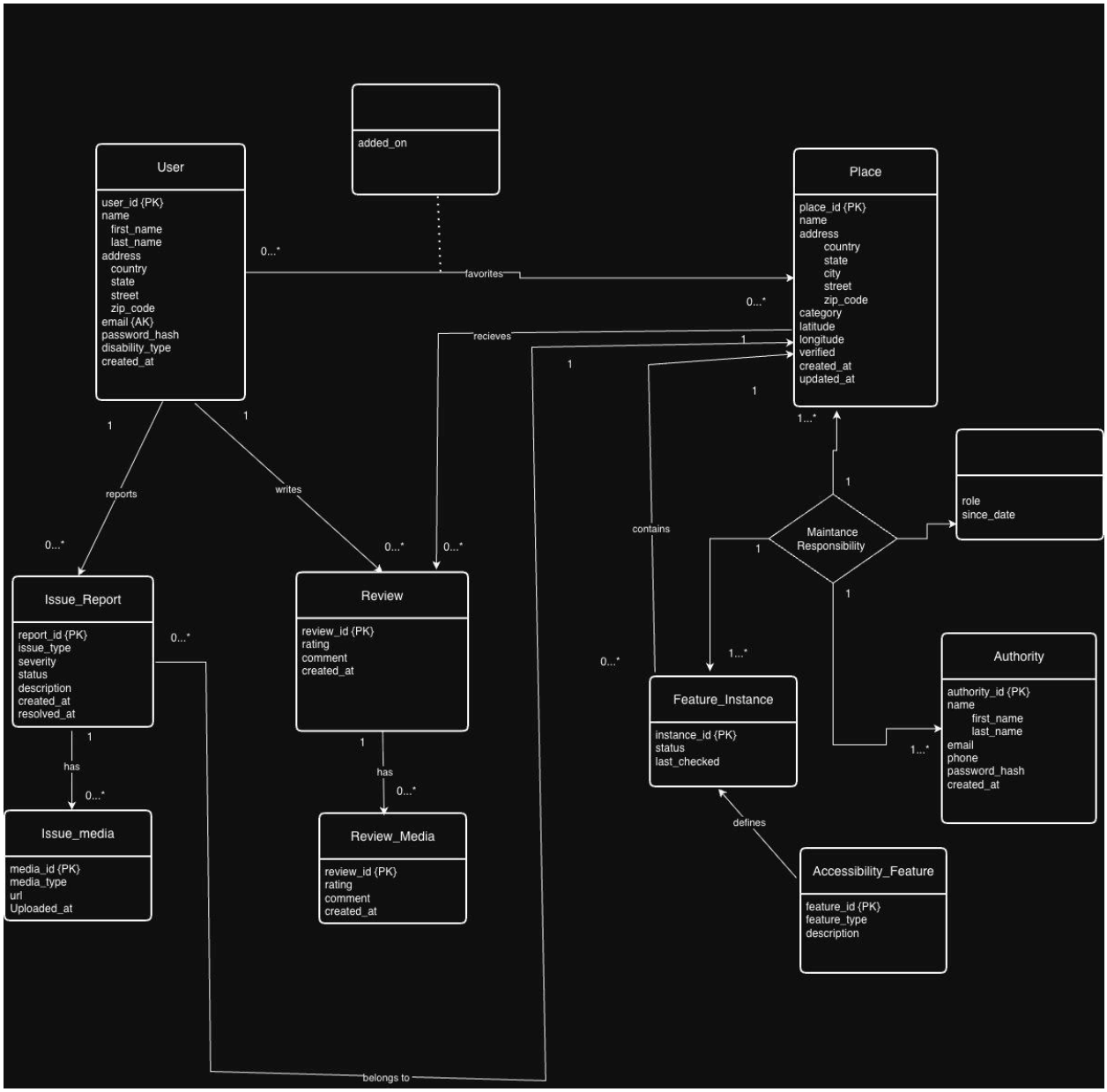
Password Security
- password_hash field for hashed password storage
- Never stores plain text passwords
- Ready for integration with hashing libraries (bcrypt, SHA-256)

Input Validation
- Email format validation in procedures
- Rating range checks (1-5)
- Enum constraints for limited value sets
- Foreign key constraints prevent orphaned records

Transaction Safety
- ROLLBACK on validation failures
- Atomic operations prevent partial updates
- Consistent state maintained even on errors

==========

3. CONCEPTUAL DESIGN (UML DIAGRAM)



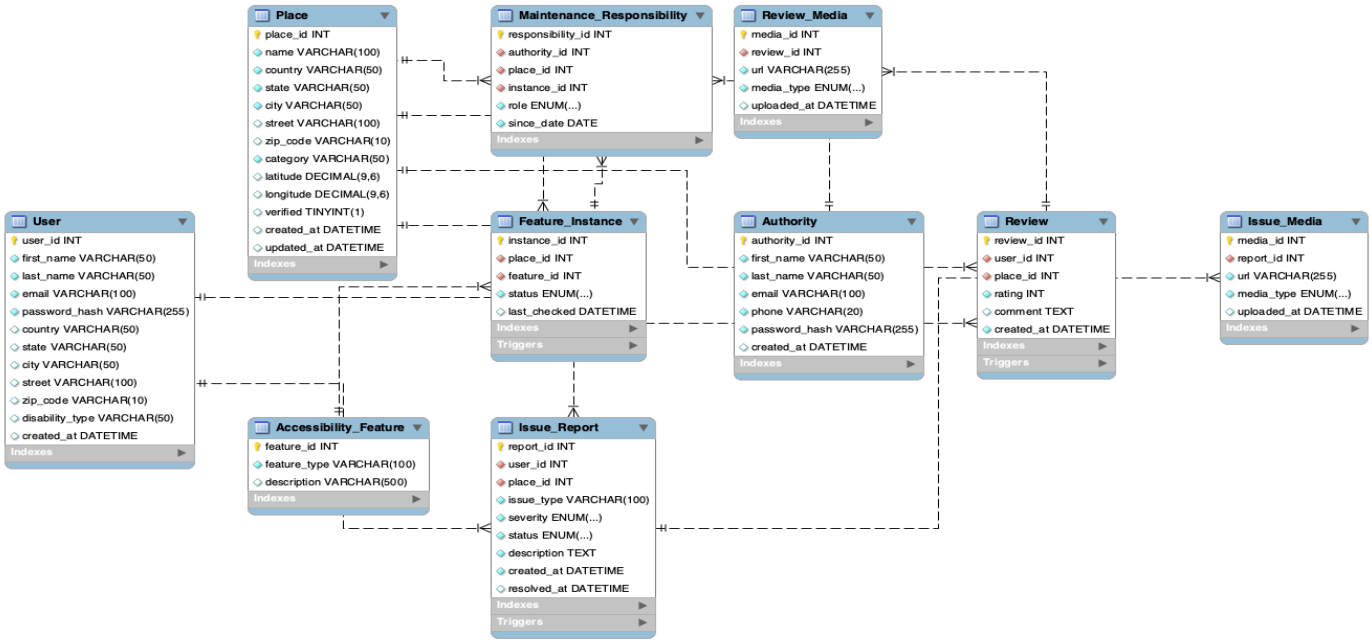
The UML diagram illustrates the entity-relationship model for the AccessMap+ database, showing all entities, attributes, and relationships between tables. Key relationships include:

- User to Review (1:N) - Users can write multiple reviews
- User to Issue_Report (1:N) - Users can report multiple issues
- Place to Feature_Instance (1:N) - Places can have multiple features
- Authority to Maintenance_Responsibility (1:N) - Authorities manage multiple features

The diagram demonstrates proper cardinality notation and identifies all primary and foreign key relationships.

==========

4. LOGICAL DESIGN (REVERSE ENGINEERING DIAGRAM)



The reverse engineering diagram shows the actual database schema as implemented in MySQL, including:

- All table structures with column names and data types
- Primary key constraints (PK)
- Foreign key relationships (FK) with referential actions
- Unique constraints
- Index definitions
- Default values and auto-increment specifications

This diagram was generated using MySQL Workbench's reverse engineering feature and represents the current production schema of the AccessMap+ database.

==========

5. FINAL USER FLOW

5.1 User Registration and Authentication

User Registration

Procedure: proc_add_review (adapted for user creation)

Method:

1. User fills registration form with first name, last name, email, password
2. System validates email format
3. System checks for duplicate email
4. Password is hashed and stored
5. User record created with unique user_id

Outcome:

- Success: User registered, redirected to login
- Error: "Email already exists" or "Invalid email format"

User Login

Method:

1. User enters email and password
2. System retrieves user by email
3. System verifies password hash
4. Session created with user_id

Outcome:

- Success: User authenticated, redirected to dashboard
- Error: "Invalid credentials"

5.2 Place Discovery and Search

Browse Places

Query: SELECT with filtering

Method:

1. User selects place by typing in a search bar
2. System retrieves places matching criteria

View Place Details

Function: get_average_rating(), count_working_features()

Method:

1. User clicks on a place
2. System displays:
   - Place information (name, address, category)
   - Average rating from reviews
   - List of accessibility features with status
   - Recent reviews from other users(by registered users)
   - Open issue reports(by registered users)

Outcome: Comprehensive place accessibility profile displayed

5.3 Review Management

Submit Review

Procedure: proc_add_review(user_id, place_id, rating, comment)

Method:

1. User navigates to place details page
2. User fills review form:
   - Rating: 1-5 stars (dropdown)
   - Comment: Text feedback (optional)
3. System validates:
   - User is logged in
   - Place exists
   - Rating is between 1-5
4. Procedure executes with error handling

Outcome:

- Success: "Review added successfully", review_id returned
- Error: "User does not exist", "Place does not exist", or "Rating must be between 1 and 5"

Update Review

Procedure: proc_add_review (with ON DUPLICATE KEY UPDATE)

Method:

1. User navigates to their existing review
2. User modifies rating or comment
3. System updates existing review using same procedure
4. Timestamp updated to current time

Outcome: Review updated, "Review updated successfully"

View User's Reviews

Query: SELECT from Review table

Method:

```
SELECT r.review_id, r.rating, r.comment, r.created_at,
       p.name AS place_name
FROM Review r
JOIN Place p ON r.place_id = p.place_id
WHERE r.user_id = ?
ORDER BY r.created_at DESC;
```

5.4 Issue Reporting

Report Accessibility Issue

Procedure: proc_create_issue(user_id, place_id, issue_type, severity, description)

Method:

1. User navigates to place page
2. User clicks "Report Issue" button
3. User fills issue report form:
   - Issue Type: Text field (e.g., "Broken Elevator")
   - Severity: Dropdown (Low, Medium, High)
   - Description: Detailed text description
4. Procedure validates user and place existence
5. Issue report created with status 'Open'

Outcome:

- Success: "Issue report created", report_id returned
- Error: "User does not exist" or "Place does not exist"

Track Issue Status

Query: SELECT from Issue_Report

Method:

1. User views their submitted issues
2. System displays:
   - Issue type and description
   - Severity level
   - Current status (Open, In Progress, Resolved, Closed)
   - Created date
   - Resolved date (if applicable)

SQL Example:

```
SELECT report_id, issue_type, severity, status, description,
       created_at, resolved_at
FROM Issue_Report
WHERE user_id = ?
ORDER BY created_at DESC;
```

5.5 Authority Operations

Assign Maintenance Responsibility

Procedure: proc_assign_responsibility(authority_id, instance_id, role, since_date)

Method:

1. Authority logs into system
2. Authority views feature instances needing assignment
3. Authority fills assignment form:
   - Feature Instance: Dropdown selection
   - Role: Maintenance, Inspection, or Cleaning
   - Since Date: Date picker
4. Procedure auto-retrieves place_id from instance
5. Responsibility assigned (or updated if duplicate)

Outcome:

- Success: "Responsibility assigned"
- Error: "Authority does not exist" or "Feature instance does not exist"

Update Feature Status

Procedure: proc_update_feature_status(instance_id, status)

Method:

1. Authority views assigned features
2. Authority updates feature status:
   - Working: Feature operational
   - Broken: Feature non-functional
   - Unavailable: Feature temporarily unavailable
3. Procedure updates status and last_checked timestamp
4. Trigger automatically updates Place.updated_at

Outcome:

- Success: "Feature status updated"
- Cascading effect: Place marked as recently updated

Resolve Issues

Procedure: proc_resolve_issue(report_id, authority_id)

Method:

1. Authority views open issues for assigned places
2. Authority selects issue to resolve
3. Procedure validates authority and report existence
4. Issue updated to 'Resolved' by authority.

Outcome:

- Success: "Issue resolved"
- Automated: Issue will auto-close after 30 days

5.6 Error Scenarios and User Feedback

Validation Errors

Scenario 1: Invalid Rating

- User attempts to submit rating of 6
- Trigger fires: trg_before_insert_review
- Error: "Rating must be between 1 and 5"
- User sees error message, form not submitted

Scenario 2: Non-existent User

- Procedure called with invalid user_id
- Validation check fails
- ROLLBACK executed
- Error: "User does not exist"
- Transaction fully reverted

Database Errors

Scenario: Database Connection Lost

- SQL exception occurs during INSERT
- EXIT HANDLER FOR SQLEXCEPTION catches error
- ROLLBACK executed
- Error: "Failed to create issue report"
- User notified to retry

==========

6. LESSONS LEARNED

6.1 Technical Expertise Gained

Through this project, we developed strong skills in database design and SQL programming. Working with MySQL 8.0, we learned to implement proper normalization (3NF) while keeping the schema practical. The biggest technical gains were:

Stored Procedures and Functions:
We implemented 5 stored procedures with comprehensive error handling using START TRANSACTION, COMMIT, and ROLLBACK. Learning to use EXIT HANDLER FOR SQLEXCEPTION was
crucial for graceful error recovery. The 3 user-defined functions we created taught us how to handle NULL values properly with COALESCE and when to mark functions as DETERMINISTIC vs non-deterministic.

Triggers and Events:
Creating 3 triggers helped us understand how to enforce business logic at the database level. We learned when to use BEFORE vs AFTER triggers and their performance implications. The 2 scheduled events we implemented taught us about automated database maintenance and the importance of the event scheduler configuration.

Foreign Keys and Constraints:
Extensive work with foreign keys taught us when to use CASCADE vs RESTRICT for ON DELETE
actions. We learned to balance data integrity with application flexibility through proper constraint design.

Performance Considerations:
We created 20+ indexes strategically on foreign keys and frequently queried columns. Understanding composite indexes for common query patterns (like city + category searches) was particularly valuable.

6.2 Insights

Time Management:
We learned that upfront planning is critical. Spending time on the ER diagram and schema design early prevented major refactoring later. However, we underestimated the time needed for error handling implementation and trigger debugging. Sample data creation also

took longer than expected due to foreign key dependencies requiring specific insertion orders.

Data Domain Understanding:
Working with accessibility mapping taught us the importance of status tracking (Working, Broken, Unavailable) for real-time information. We realized that accessibility scoring needs multiple factors (features, ratings, issues) and that maintenance relationships are complex enough to require ternary tables. The temporal nature of accessibility data (features break and get fixed) required careful design of timestamp handling.

Key Challenges:
Media storage presented an interesting challenge - we decided to store URLs rather than binary data, requiring external storage solutions. We also recognized that high-volume data like reviews and issues would need archiving strategies, which led to our auto-closure event implementation.

6.3 Alternative Designs Considered

We considered several alternative approaches during development:

NoSQL vs SQL:
We initially considered MongoDB for flexibility with media and dynamic attributes, but chose MySQL because our data has clear relational structure, needs ACID properties for consistency, and requires complex JOIN queries. The lesson: not every project needs NoSQL.

Media Table Structure:
We debated having separate tables for Review_Images, Review_Videos, Issue_Images, and Issue_Videos but consolidated into two tables (Review_Media, Issue_Media) with an ENUM type field. This simplified the schema significantly while maintaining type safety.

Performance vs Normalization:
We considered storing average_rating directly in the Place table instead of calculating it via function. We rejected this because it would violate 3NF and require complex triggers to maintain consistency. The calculated function approach with proper indexing proved sufficiently performant.

Deletion Strategy:
We implemented a mixed approach: soft deletes for Issue_Reports (via status='Closed'), hard deletes with CASCADE for User/Review relationships, and RESTRICT for Accessibility_Feature to prevent accidental deletion. Different tables need different

strategies based on business requirements.

6.4 Code Issues and Limitations

Event Scheduler Dependency:
Our events require the MySQL event_scheduler to be enabled globally, which some installations disable by default. We included SET GLOBAL event_scheduler = ON in the dump file and documented this requirement in the README.

Email Validation:
Email format validation exists only in procedures, not as a CHECK constraint (MySQL 5.7 limitation). Direct SQL console INSERT operations bypass this validation, so we rely on UNIQUE constraints and application-layer validation.

Timezone Handling:
We use DATETIME fields that don't store timezone information. This could cause confusion for multi-timezone deployments. For a global application, we'd need to switch to TIMESTAMP or add timezone columns.

Review Uniqueness:
Our UNIQUE(user_id, place_id, created_at) constraint allows multiple reviews at different times, which enables update history but could theoretically allow spam. We decided this was acceptable with application-layer rate limiting.

Working Features Beyond Initial Plan:
We exceeded our initial goals by implementing comprehensive error handling in all procedures (originally planned simple SIGNAL), automated maintenance events (planned for manual administration), and a full ternary relationship for maintenance responsibilities (initially planned as simpler two-way assignment).

==========

7. FUTURE WORK

7.1 Planned Uses of the Database

Web Application Development:
Our primary plan is to build a full-stack web application using Flask or Django. The application would provide user registration, place search, review submission, and issue reporting through a user-friendly interface. We'd connect to the database using

SQLAlchemy ORM and create RESTful API endpoints for each stored procedure. Key features would include role-based access (regular users vs authorities), interactive maps for place discovery.

Mobile Application:
We're considering developing iOS and Android apps using React Native or Flutter. The mobile app would use the same MySQL database through a REST API backend and implement offline-first architecture for better user experience. Important mobile-specific features would include GPS-based place discovery, camera integration for issue photos, and push notifications for accessibility alerts.

Data Analytics and Reporting:
The database structure supports comprehensive analytics. We plan to generate monthly accessibility reports showing trends by city, track issue resolution rates by authority, and identify places needing improvements. This could be implemented using data warehousing techniques with nightly ETL processes and visualization through Tableau or Power BI. Predictive analytics could help forecast maintenance needs.

Open Data Platform:
We'd like to provide a public API for third-party developers to build accessibility apps using our data. This would include weekly exports of verified places and features in JSON/CSV formats while maintaining user privacy through data anonymization. The platform could integrate with city planning departments and public transportation systems to share accessibility information more broadly.

7.2 Potential Areas for Added Functionality

Enhanced Accessibility Scoring:
Currently, we have basic scoring logic. We want to expand this with weighted scoring based on disability type (wheelchair users might prioritize ramps while visually impaired users prioritize braille signage), implement time-decay for aging verifications, and add confidence scores based on number of confirmations. This would require new tables for disability_weight and verification_confidence.

User Reputation and Gamification:
To encourage participation, we'd add a reputation system tracking user contributions. This would include a User_Reputation table storing scores, review counts, and badges earned. We'd implement badges like "5-Star Reviewer" or "Issue Hunter" and create leaderboards for top contributors per city. Triggers would automatically update reputation on review/report creation.

Notification System:
Users should be alerted when something changes at their favorited places. We'd create a Notification table storing alerts for issue resolutions, new reviews, and feature status changes. Triggers would generate notifications on key events, and users could mark them as read. Authorities would receive notifications about new issues in their jurisdictions.

Route Planning:
An advanced feature would be accessible route planning between locations. This would require Route and Route_Segment tables to store path information. The system could avoid broken features, prefer highly rated places, and suggest alternative routes. A function would calculate overall route accessibility scores considering all segments.

Feature Request System:
Users could request new accessibility features at places through a Feature_Request table. Other users would vote on requests, authorities would review and approve them, and we'd track funding and installation progress. Completed requests would convert to actual Feature_Instances.

Additional Enhancements:
- Full-text search on place names/descriptions with FULLTEXT indexes
- Geographic radius search using geospatial functions
- Multilingual support with translation tables for global expansion
- Historical tracking of feature changes through Feature_Instance_History table
- Integration with Google Maps, OpenStreetMap, and transit APIs
- Crowdsourced verification system where multiple user confirmations auto-verify features
- Accessibility certification program awarding Bronze/Silver/Gold badges to highly accessible places

7.3 Justification for Extensive Future Work

The AccessMap+ database has significant potential because accessibility mapping is an underserved area with real social impact. Our modular design with stored procedures and triggers provides a solid foundation that can support these enhancements without major schema changes. The core tables are normalized and flexible enough to accommodate new features through additional junction tables and relationships rather than restructuring existing ones.

==========